Linear Search and Binary Search

Data Structures and Algorithms (DSA) — Theory Paper Notes

1. Searching

Definition: Searching is the process of finding a particular element in a collection of data. The element that we want to find is called the *key* or *search key*.

Searching is used to locate data, check whether an element exists, and retrieve the required information. Two common searching techniques are Linear Search and Binary Search.

2. Linear Search

Definition

Linear Search is a searching technique in which each element of a list is checked one by one, starting from the first element, until the required element is found or the end of the list is reached. It is also called **Sequential Search**.

Working of Linear Search

1. Start from the first element of the list.
2. Compare the current element with the search key.
3. If the element matches the key, report its position.
4. If it does not match, move to the next element.
5. Repeat until the element is found or the list ends.
6. If all elements are checked and no match is found, report that the key is not present.

Example

Consider the list: **10, 25, 30, 45, 50**
Search key: 45

First, compare 45 with 10. They are not equal. Next, compare it with 25; they are not equal. Then compare it with 30; they are not equal. Finally, compare it with 45. The element is found.

Result: 45 is the fourth element in the list (index 3 when indexing starts from 0).

Algorithm: Linear Search

```
LINEAR_SEARCH(A, n, key)
1. Start
2. For i = 0 to n - 1
3.    If A[i] == key, return i
4. Return -1 if the key is not found
5. Stop
```

Here, A represents the array, n is the number of elements, and key is the value to be searched. Returning -1 indicates that the element is not present.

Advantages of Linear Search

- It is simple and easy to understand and implement.
- It works with both sorted and unsorted data.

- It does not require the elements to be arranged in any particular order.
- It is suitable for small lists.

Disadvantages of Linear Search

- It may need to check every element before reaching a result.
- It becomes inefficient when the list contains a large number of elements.
- If the key is at the end of the list or is absent, many comparisons may be required.

Time and Space Complexity of Linear Search

Best case — O(1): The key is the first element, so only one comparison is needed.
Average case — O(n): A number of elements must be checked on average.
Worst case — O(n): The key is the last element or is not present, so all elements may be checked.
Space complexity — O(1): The iterative method uses a constant amount of extra space.

3. Binary Search

Definition

Binary Search is a searching technique in which the search interval is repeatedly divided into two halves until the required element is found or no elements remain to be searched.

Important condition: Binary Search requires the data to be arranged in sorted order.

Working of Binary Search

1. Identify the first and last positions of the search interval.
2. Find the middle position of the interval.
3. Compare the middle element with the search key.
4. If they are equal, the element is found.
5. If the key is smaller than the middle element, continue searching in the left half.
6. If the key is greater than the middle element, continue searching in the right half.
7. Repeat until the element is found or the interval becomes empty.

Example

Consider the sorted list: **10, 20, 30, 40, 50, 60, 70**
Search key: 60

Step 1: The middle element is 40. Since 60 is greater than 40, ignore the left half and search the right half.

Step 2: The remaining elements are 50, 60, and 70. Their middle element is 60, which matches the search key.

Result: 60 is the sixth element in the list (index 5 when indexing starts from 0).

Important Formula

The middle position is calculated as:
mid = (low + high) / 2

In programming, integer division is used. A safer equivalent formula is:
mid = low + (high - low) / 2

low: index of the first element in the current interval.
high: index of the last element in the current interval.
mid: index of the middle element.

Algorithm: Binary Search

```
BINARY_SEARCH(A, n, key)
1. Start
2. Set low = 0 and high = n - 1
3. While low <= high
4.    mid = low + (high - low) / 2
5.    If A[mid] == key, return mid
6.    Else if key < A[mid], set high = mid - 1
7.    Else set low = mid + 1
8. Return -1 if the key is not found
9. Stop
```

The array A contains the sorted elements, n is the number of elements, and key is the value to be searched. Returning -1 means the key is not present.

Advantages of Binary Search

- It is efficient for searching large, sorted arrays.
- It reduces the search interval by half in each step.
- It usually needs far fewer comparisons than Linear Search for large sorted arrays.

Disadvantages of Binary Search

- The data must be sorted before the search can be performed.
- Sorting an unsorted collection may require additional time.
- It is less suitable for linked lists because accessing the middle element directly is not efficient.
- Its logic is more complex than Linear Search.

Time and Space Complexity of Binary Search

Best case — O(1): The middle element matches the key on the first comparison.
Average case — O(log n): The search interval is repeatedly halved.
Worst case — O(log n): The algorithm continues halving the interval until the key is found or the interval becomes empty.
Space complexity — O(1): The iterative algorithm uses a constant amount of extra space.

4. Important Theory-Paper Questions

- Define searching and explain Linear Search with an example.
- Explain the working and algorithm of Linear Search.
- Define Binary Search and explain its working with an example.

- Write the Binary Search algorithm and explain low, high, and mid.
- Explain the advantages, disadvantages, and time complexity of Linear Search.
- Explain the advantages, disadvantages, and time complexity of Binary Search.
- Differentiate between Linear Search and Binary Search.

Exam tip: For a short-answer question, write the definition and working. For a longer answer, include the algorithm, a worked example, advantages, disadvantages, and time and space complexity.